

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
федеральное государственное бюджетное образовательное учреждение высшего образования  
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

## Отчет

по учебной практике

УП.02.01 Технология разработки программного обеспечения  
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции  
программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся \_\_\_\_\_ **Правденкова Полина Игоревна**  
(подпись) (фамилия, имя, отчество)

Группы ИПо-32

Руководитель практической подготовки от профильной организации

Преподаватель  
(должность)

\_\_\_\_\_ **Берестов Дмитрий Борисович**  
(подпись) (фамилия, имя, отчество)

МП «26» июня 2026 года

Руководитель практической подготовки от института

\_\_\_\_\_ **Берестов Дмитрий Борисович**  
(подпись) (фамилия, имя, отчество)

«27» июня 2026 года

Пермь, 2026 год

# ОГЛАВЛЕНИЕ

|                                                                                                                                                       |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ .....                                                                                                                                        | 3  |
| Раздел 1. Разработка требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент..... | 5  |
| 1.1    Сбор информации о существующем состоянии продукта: .....                                                                                       | 5  |
| 1.2    Разработка технического задания .....                                                                                                          | 5  |
| Раздел2. Интеграция модулей в программное обеспечение .....                                                                                           | 7  |
| 2.1. Процесс разработки программного обеспечения .....                                                                                                | 7  |
| 2.1.1.    Выбор модели и инструментов разработки .....                                                                                                | 7  |
| 2.1.2.    Разработка программного модуля .....                                                                                                        | 7  |
| 2.1.3.    Разработка пользовательского интерфейса.....                                                                                                | 9  |
| Раздел 3. Отладка программного модуля с использованием специализированных программных средств.....                                                    | 14 |
| 3.1. Процесс отладки .....                                                                                                                            | 14 |
| Раздел 4. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения .....                                            | 16 |
| 4.1. Процесс тестирования.....                                                                                                                        | 16 |
| Раздел 5. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования.....                                      | 20 |
| 5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования (описать процесс).....                 | 20 |
| 5.1.1.    Виды инспекций, применявшиеся в ходе разработки.....                                                                                        | 20 |
| 5.1.2.    Стандарты кодирования, применявшиеся в проекте. ....                                                                                        | 20 |
| 5.1.3.    Программные средства для проведения инспекции. ....                                                                                         | 21 |
| Заключение.....                                                                                                                                       | 22 |
| Список используемых источников .....                                                                                                                  | 24 |

## **ВВЕДЕНИЕ**

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Учебная практика направлена на формирование у обучающихся первоначальных практических профессиональных умений в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем

В настоящей работе представлен отчет о прохождении учебной практики студента-практиканта. Основными видами деятельности являлись: Разработка программного обеспечения; выполнение отладки программного модуля с использованием специализированных программных средств; осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения; уметь производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент;
2. Выполнять интеграцию модулей в программное обеспечение;
3. Выполнять отладку программного модуля с использованием специализированных программных средств;

4. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения.
5. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования
6. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;
7. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности
8. Планировать и реализовывать собственное профессиональное и личностное развитие;
9. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами
10. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.
11. Использовать информационные технологии в профессиональной деятельности
12. Пользоваться профессиональной документацией на государственном и иностранном языках

## **РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ**

### **1.1 Сбор информации о существующем состоянии продукта:**

На начальном этапе продукт был в форме представления. Сам проект подразумевал использование библиотеки классов и WPF интерфейса. Идея продукта заключалась в создании, управлении, удалении задач. Были примерно понятны функции и программное заполнение. В идеале для продукта можно использовать базу данных, но сам он подразумевает загрузку данных через файл, также это позволяет приложению работать в офлайн режиме.

В готовом состоянии продукт должен представлять из себя WPF-приложение с библиотекой классов. Необходимо создать окна для управления задачами, где главным окном будет отображение задач с поиском, сортировкой и фильтрацией. На главном окне также расположены снизу будут кнопки для взаимодействия и управления. Отдельно будут предоставляться окна добавления, редактирования и статистики. Дополнительные окна будут открываться при взаимодействии с главным окном, либо через клик мыши, либо через простое нажатие кнопки. В приложении должна присутствовать возможность загружать данные через файлы JSON или XML, также сохранение задач в одном из форматов.

### **1.2 Разработка технического задания**

Техническое задание на разработку программного модуля «Менеджер задач» включает следующие основные требования:

- Разработать приложение для управления задачами с возможностью добавления, просмотра, редактирования и удаления задач;
- Реализовать фильтрацию задач по статусу («Новая», «В процессе», «Завершена»);
- Реализовать поиск задач по названию или описанию;

- Реализовать сортировку задач по приоритету и сроку выполнения;
- Добавить возможность пометки задачи как «Важной» с визуальным отображением;
- Реализовать статистику по задачам (количество по статусам и просроченные);
- Обеспечить сохранение и загрузку данных в форматах JSON и XML;
- Основную логику разместить в динамической библиотеке классов, пользовательский интерфейс выполнить в виде WPF-приложения;
- Написать модульные тесты для классов библиотеки.

## РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

### 2.1. Процесс разработки программного обеспечения

#### 2.1.1. Выбор модели и инструментов разработки

Для проекта была выбрана итерационная модель, так как она позволяет поэтапно разрабатывать и тестировать отдельные модули, что удобно при разработке приложения с чётко выделенными компонентами (модель, логика, UI).

Сам проект разработан в программе Visual Studio 2022, в решение входили проекты библиотеки классов, WPF-приложения, MS-Test.

Где ролью интерфейса и основного взаимодействия с программой является WPF-приложение. Вся логика и функции, события происходят через библиотеку классов, которая и проходит модульные тесты через MS-Test.

#### 2.1.2. Разработка программного модуля

Класс Task с полями: Guid id – уникальный идентификатор задачи, string name – название задачи, string description – описание задачи, Priority1 priority – приоритет задачи (Низкий, Средний, Высокий), DateTime dueDate – срок выполнения, Status1 status – статус задачи (Новая, ВПроцессе, Завершена), bool isImportant – пометка "Важная"; и методами: Guid GetId() – получить идентификатор, void SetName(string name) – установить название, string GetName() – получить название, void SetDescription(string description) – установить описание, string GetDescription() – получить описание, void SetPriority(Priority1 priority) – установить приоритет, Priority1 GetPriority() – получить приоритет, void SetDueDate(DateTime dueDate) – установить срок выполнения, DateTime GetDueDate() – получить срок выполнения, void SetStatus(Status1 status) – установить статус, Status1 GetStatus() – получить статус, void SetIsImportant(bool isImportant) – установить пометку "Важная", bool GetIsImportant() – получить пометку "Важная", string GetImportantMark()

– получить строковое представление пометки. И enum Priority1 – Низкий, Средний, Высокий; enum Status1 – Новая, ВПроцессе, Завершена.

Класс Manager с полями: List<Task> \_tasks – список задач; и методами: List<Task> GetAll() – получить список всех задач, Task GetById(Guid id) – получить задачу по идентификатору, void Add(Task task) – добавить новую задачу, void AddRange(List<Task> tasks) – добавить несколько задач, void Edit(Task updatedTask) – редактировать существующую задачу, void Delete(Guid id) – удалить задачу по идентификатору, void Clear() – очистить список всех задач.

Класс Stats с методами: Dictionary<Status1, int> GetStatusCounts(List<Task> tasks) – получить количество задач по каждому статусу, int GetOverdueCount(List<Task> tasks) – получить количество просроченных задач, int GetTotalCount(List<Task> tasks) – получить общее количество задач, Dictionary<string, int> GetAllStats(List<Task> tasks) – получить всю статистику в виде словаря, bool IsOverdue(Task task) – проверить, просрочена ли задача.

Класс Storage с методами: void SaveToJson(List<Task> tasks, string filePath) – сохранить задачи в JSON-файл, List<Task> LoadFromJson(string filePath) – загрузить задачи из JSON-файла, void SaveToXml(List<Task> tasks, string filePath) – сохранить задачи в XML-файл, List<Task> LoadFromXml(string filePath) – загрузить задачи из XML-файла, List<Task> LoadFromFile(string filePath) – загрузить задачи из файла (определяет формат по расширению), void SaveToFile(List<Task> tasks, string filePath) – сохранить задачи в файл (определяет формат по расширению).

Класс Sorter с методами: List<Task> SortByPriority(List<Task> tasks) – отсортировать задачи по приоритету (по убыванию), List<Task> SortByDueDate(List<Task> tasks) – отсортировать задачи по сроку выполнения (по возрастанию), List<Task> SortByPriorityAndDueDate(List<Task> tasks) – отсортировать задачи сначала по приоритету, затем по сроку.



Класс `Filter` с методами: `List<Task> FilterByStatus(List<Task> tasks, Status1 status)` – отфильтровать задачи по статусу, `List<Task> FilterByStatuses(List<Task> tasks, List<Status1> statuses)` – отфильтровать задачи по нескольким статусам, `List<Task> FilterByImportant(List<Task> tasks)` – отфильтровать задачи с пометкой "Важная", `List<Task> FilterByPriority(List<Task> tasks, Priority1 priority)` – отфильтровать задачи по приоритету.

Класс `Searcher` с методами: `List<Task> Search(List<Task> tasks, string query)` – выполнить поиск по названию или описанию задачи, `List<Task> SearchByExactName(List<Task> tasks, string name)` – выполнить поиск по точному совпадению названия.

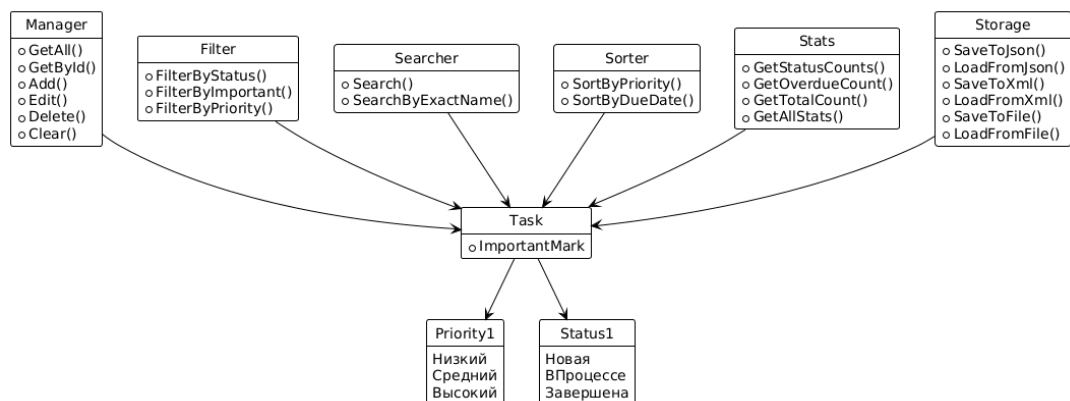


Рисунок 1 – Диаграмма классов

### 2.1.3. Разработка пользовательского интерфейса

Главное окно является основным окном приложения и предназначено для управления списком задач. В верхней части окна расположен заголовок «Менеджер задач», который информирует пользователя о назначении приложения.

Под заголовком находится панель с элементами управления для работы со списком задач, включающая поле поиска для ввода ключевых слов – поиск осуществляется по названию или описанию задачи автоматически при вводе текста. Кроме того, на панели присутствует выпадающий список сортировки, позволяющий отсортировать задачи по одному из критериев:

«Без сортировки», «По приоритету», «По сроку», и выпадающий список фильтра для фильтрации задач по статусу: «Все», «Новая», «В процессе», «Завершена».

Центральную часть окна занимает список всех задач, каждая из которых отображается в виде карточки, содержащей название задачи (выделено жирным шрифтом), описание (отображается серым цветом), приоритет (Низкий, Средний, Высокий), срок выполнения (в формате ДД.ММ.ГГГГ), статус (Новая, В процессе, Завершена), а также пометку «Важная», отображаемую звёздочкой. Для просмотра подробной информации или редактирования задачи достаточно выполнить двойной клик по карточке задачи.

В нижней части окна расположены кнопки для выполнения основных операций с задачами: «Добавить» – открывает окно добавления новой задачи; «Редактировать» – открывает окно редактирования выбранной задачи; «Удалить» – удаляет выбранную задачу после подтверждения; «Статистика» – открывает окно со статистикой по задачам; «Сохранить» – сохраняет все задачи в файл (JSON или XML) с возможностью выбора места сохранения; «Загрузить из файла» – загружает задачи из ранее сохранённого файла.

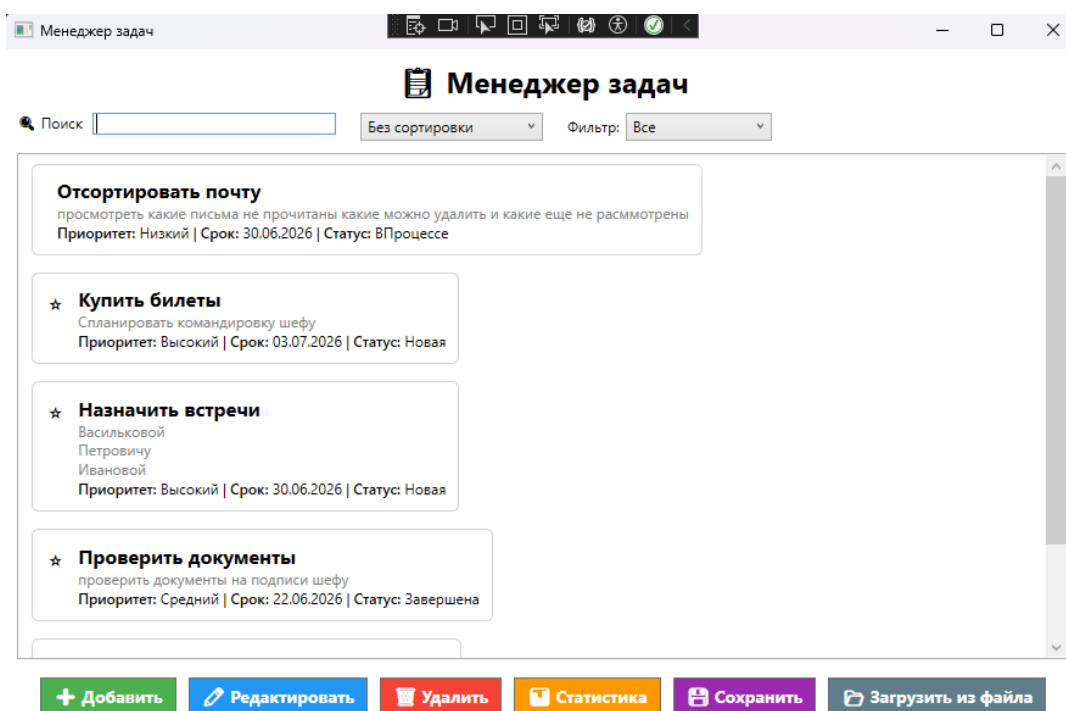


Рисунок 2 – Главное окно

Окно добавления задачи предназначено для создания новой задачи и представляет собой небольшую диалоговую форму со следующими полями ввода:

- название (обязательное поле);
- описание (поддерживает многострочный ввод);
- приоритет (выпадающий список с вариантами «Низкий», «Средний», «Высокий»);
- срок выполнения (элемент выбора даты DatePicker);
- статус (выпадающий список с вариантами «Новая», «В процессе», «Завершена»).

Дополнительно в окне присутствует чекбокс «Пометить как важную», позволяющий отметить задачу как важную, а также кнопка «Сохранить» для создания задачи и закрытия окна, и кнопка «Отмена» для закрытия без сохранения. При попытке сохранить задачу без заполненного названия пользователю выводится предупреждение.

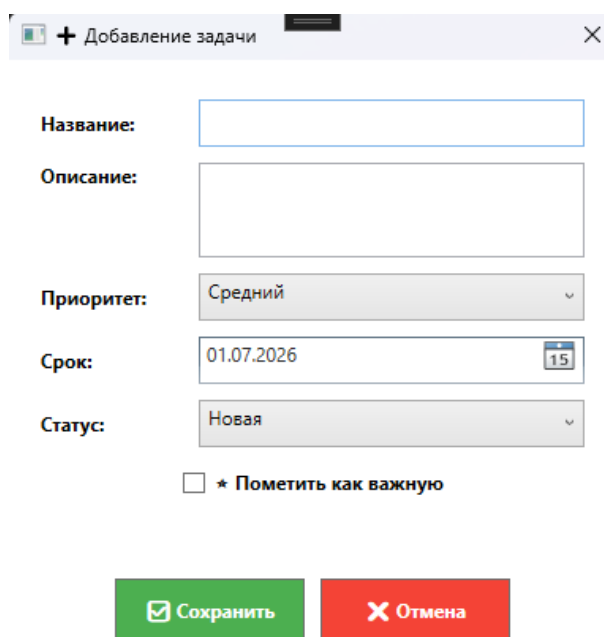


Рисунок 3 – Окно добавления

Окно редактирования задачи визуально похоже на окно добавления, но предназначено для изменения существующей задачи. Оно содержит те же поля, что и окно добавления, однако они изначально заполнены данными

выбранной задачи: название, описание, приоритет, срок выполнения, статус и пометка «Важная» подгружаются из выбранной задачи, и пользователь может изменить любые поля по своему усмотрению. Кнопка «Сохранить» применяет изменения к задаче и закрывает окно, а кнопка «Отмена» закрывает окно без сохранения изменений.

Редактирование задачи

**Редактирование задачи**

Название: Отсортировать почту

Описание: просмотреть какие письма не прочитаны  
какие можно удалить и какие еще не  
рассмотрены

Приоритет: Низкий

Срок: 30.06.2026

Статус: ВПроцессе

☐ \* Пометить как важную

Сохранить Отмена

Рисунок 4 – Окно редактирования

Окно статистики отображает сводную информацию по всем задачам в виде списка, включающего общее количество задач, количество задач со статусом «Новая», количество задач со статусом «В процессе», количество задач со статусом «Завершена», а также количество просроченных задач — тех, у которых срок выполнения меньше текущей даты и статус не «Завершена». Информация представлена в виде пар «название показателя — значение», где значение выделено синим цветом и крупным шрифтом для наглядности.

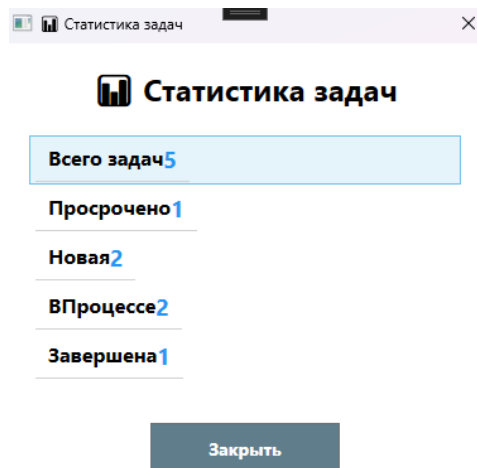


Рисунок 5 – Окно статистики

При нажатии кнопок «Сохранить» или «Загрузить из файла» в главном окне открываются стандартные диалоговые окна Windows. При сохранении в файл открывается диалог `SaveFileDialog`, где пользователь может выбрать папку для сохранения, указать имя файла и выбрать формат: JSON или XML, после чего выводится уведомление об успешном завершении операции. При загрузке из файла открывается диалог `OpenFileDialog`, пользователь выбирает ранее сохранённый файл в формате JSON или XML, и задачи из файла добавляются к существующему списку без потери текущих данных, после чего выводится уведомление о количестве загруженных задач. Кроме того, при закрытии главного окна приложения все задачи автоматически сохраняются в файл `tasks.json` в папке с программой без дополнительных диалогов, а при следующем запуске приложения данные автоматически загружаются из этого файла; если файл отсутствует, приложение запускается с пустым списком задач.

## **РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММ- НЫХ СРЕДСТВ**

### **3.1. Процесс отладки**

Отладка разработанного программного модуля проводилась с использованием встроенных средств Visual Studio. В процессе работы применялись такие инструменты, как пошаговое выполнение кода с использованием клавиш F10 (шаг с обходом) и F11 (шаг с заходом), точки останова (Breakpoints), позволяющие приостанавливать выполнение программы в определённых местах для анализа состояния переменных, а также окна наблюдения (Watch), с помощью которых отслеживались изменения значений ключевых переменных в процессе выполнения программы.

В ходе отладки были проверены все основные сценарии работы приложения: добавление новой задачи, редактирование существующей, удаление, фильтрация по статусу, поиск по названию и описанию, сортировка по приоритету и сроку выполнения, а также сохранение и загрузка данных в форматах JSON и XML.

В процессе отладки был выявлен ряд проблем, которые потребовали доработки кода.

Первая проблема заключалась в том, что при закрытии приложения все добавленные задачи терялись, так как изначально автосохранение не было реализовано. Пользователь мог сохранить данные только вручную через кнопку «Сохранить», однако в случае случайного закрытия окна все несохранённые изменения безвозвратно терялись. Для решения данной проблемы было принято решение реализовать автоматическое сохранение задач при закрытии главного окна приложения. Для этого в обработчике события Closing главного окна был добавлен вызов метода SaveToJson, который сохраняет все текущие задачи в файл tasks.json в папке с

исполняемым файлом приложения. Благодаря этому данные автоматически сохраняются без дополнительных действий со стороны пользователя, а при следующем запуске приложения они загружаются автоматически, если файл существует. Если файл отсутствует, приложение запускается с пустым списком задач, что также было проверено в процессе отладки.

Вторая проблема была связана с фильтрацией задач по статусу. В процессе отладки выяснилось, что значения статусов в выпадающем списке фильтрации и значения в перечислении `Status1` имеют несоответствие: в интерфейсе использовалось название «В процессе», а в перечислении – «ВПроцессе» (без пробела). Это приводило к возникновению ошибки при попытке отфильтровать задачи по данному статусу, так как программа не могла найти соответствующее значение в перечислении. Проблема была решена путём приведения названий к единому формату – в выпадающем списке фильтрации значение было изменено на «ВПроцессе», что обеспечило корректное сопоставление с перечислением.

Третья проблема возникла при тестировании модульных тестов для класса `Storage`. Некоторые тесты падали из-за того, что файлы, создаваемые в процессе тестирования, не удалялись после завершения тестов, что приводило к конфликтам при повторном запуске. Для устранения этой проблемы в тестовом классе был добавлен метод с атрибутом `TestCleanup`, который гарантированно удаляет временные файлы после каждого теста.

В результате проведённой отладки все выявленные проблемы были устранены, а работоспособность приложения была подтверждена на всех основных сценариях использования. Программа корректно обрабатывает добавление, редактирование, удаление, фильтрацию, поиск, сортировку, сохранение и загрузку задач, а также автоматически сохраняет данные при закрытии приложения и восстанавливает их при следующем запуске.

## **РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ**

### **4.1. Процесс тестирования**

В рамках тестирования разработанной библиотеки классов были созданы семь тестовых классов, покрывающих все основные функциональные модули приложения.

Тестовый класс `TaskTests` проверяет корректность работы модели данных `Task`. В ходе тестирования проверяется, что конструкторы класса правильно устанавливают значения свойств, а также что свойство `ImportantMark` корректно отображает пометку «Важная». Тесты подтверждают, что при создании задачи через конструктора по умолчанию ей автоматически присваиваются уникальный идентификатор, статус «Новая», приоритет «Средний», а также срок выполнения, установленный на 7 дней вперед. Дополнительно проверяется, что при создании задачи с параметрами все переданные значения корректно сохраняются в соответствующих полях, а свойство `ImportantMark` возвращает значок звезды, если задача отмечена как важная, и пустую строку в противном случае.

Тестовый класс `ManagerTests` проверяет все основные операции управления задачами, реализованные в классе `Manager`. Проверяется, что метод `Add` корректно добавляет новую задачу в список, а при попытке добавить `null` выбрасывается исключение. Метод `AddRange` проверяется на корректное добавление нескольких задач одновременно, а также на обработку `null` с выбрасыванием исключения. Метод `GetAll` подтверждает, что возвращается полный список всех добавленных задач. Метод `GetById` проверяется как на существующую задачу (возвращает правильный объект), так и на несуществующую (возвращает `null`). Метод `Edit` проверяет, что изменения корректно применяются к существующей задаче, а при попытке редактирования несуществующей



задачи выбрасывается соответствующее исключение. Метод Delete проверяется на удаление существующей задачи и на попытку удаления несуществующей задачи с выбрасыванием исключения. Метод Clear подтверждает, что все задачи удаляются из списка.

Тестовый класс FilterTests проверяет корректность работы всех методов фильтрации. Метод FilterByStatus проверяет, что фильтрация по статусу возвращает только те задачи, которые имеют указанный статус. Метод FilterByStatuses проверяет фильтрацию по нескольким статусам одновременно, возвращая задачи, соответствующие любому из переданных статусов. Метод FilterByImportant проверяет, что возвращаются только те задачи, которые имеют пометку «Важная». Метод FilterByPriority проверяет фильтрацию задач по указанному приоритету. Все методы также проверяются на корректную работу с пустым списком задач – в этом случае они должны возвращать пустой список.

Тестовый класс SorterTests проверяет корректность работы всех методов сортировки. Метод SortByPriority проверяет, что задачи сортируются по убыванию приоритета, то есть сначала идут задачи с высоким приоритетом, затем со средним и в конце с низким. Метод SortByDueDate проверяет, что задачи сортируются по возрастанию даты выполнения, то есть сначала отображаются задачи с ближайшим сроком. Метод SortByPriorityAndDueDate проверяет комбинированную сортировку – сначала по убыванию приоритета, а затем по возрастанию даты выполнения внутри каждой группы приоритетов. Все методы также проверяются на корректную работу с пустым списком задач.

Тестовый класс SearcherTests проверяет корректность работы всех методов поиска. Метод Search проверяет, что поиск по ключевому слову находит задачи, в названии или описании которых содержится данное слово, при этом проверяется регистронезависимость поиска – запрос в любом регистре должен находить соответствующие задачи. Метод Search также проверяется на пустой или состоящий из пробелов запрос, который должен

возвращать все задачи без фильтрации. Метод `SearchByExactName` проверяет точное совпадение с названием задачи и также подтверждает регистронезависимость. Все методы проверяются на корректную работу с пустым списком задач.

Тестовый класс `StatsTests` проверяет корректность работы всех методов сбора статистики. Метод `GetStatusCounts` проверяет, что корректно подсчитывается количество задач в каждом статусе. Метод `GetOverdueCount` проверяет, что правильно вычисляется количество просроченных задач. Метод `GetTotalCount` подтверждает, что возвращается общее количество задач в списке. Метод `GetAllStats` проверяет, что словарь со статистикой содержит все необходимые ключи и что значения соответствуют реальным данным. Метод `IsOverdue` проверяет определение просроченности для отдельной задачи. Все методы также проверяются на корректную работу с пустым списком задач, при этом количество задач в каждом статусе должно быть равно нулю.

Тестовый класс `StorageTests` проверяет корректность работы всех методов сохранения и загрузки данных. Для каждого теста создаются уникальные временные файлы в системной папке, чтобы избежать конфликтов между тестами. Метод `SaveToJson` проверяет, что файл в формате JSON успешно создаётся и не является пустым, а метод `LoadFromJson` проверяет, что данные корректно загружаются из JSON-файла и все свойства задач сохраняются без потерь. Аналогично проверяются методы `SaveToXml` и `LoadFromXml` для работы с XML-форматом. Метод `LoadFromFile` проверяет универсальную загрузку, которая автоматически определяет формат файла по расширению, а метод `SaveToFile` проверяет универсальное сохранение. Методы также проверяют выбрасывание исключений при попытке загрузки из несуществующего файла и при попытке загрузки или сохранения в неподдерживаемом формате. Кроме того, тест `SaveAndLoad_ShouldPreserveAllProperties` проверяет, что при сохранении

и последующей загрузке все свойства задачи сохраняются без искажений, включая дату выполнения. Для обеспечения чистоты между тестами используется метод, помеченный атрибутом [TestCleanup], который автоматически удаляет все временные файлы после каждого теста.

В сумме все тестовые классы содержат 57 тестов, которые покрывают все основные функциональные возможности библиотеки TaskManager.Library. Это позволяет обеспечить высокое качество кода и минимизировать вероятность возникновения ошибок при дальнейшем развитии приложения.

## **РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ**

### **5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования (описать процесс)**

Инспектирование компонентов программного обеспечения проводилось на всех этапах разработки приложения для управления задачами с целью обеспечения соответствия кода стандартам кодирования, повышения его читаемости и упрощения дальнейшей поддержки.

#### **5.1.1. Виды инспекций, применявшиеся в ходе разработки.**

В процессе работы над проектом использовались два вида инспекций. Во-первых, автоматическая инспекция, выполняемая непосредственно в среде разработки Visual Studio. Встроенные анализаторы кода непрерывно проверяли написанный код в реальном времени, подсвечивая нарушения правил стиля и качества прямо в редакторе и предлагая варианты исправлений. Во-вторых, ручная инспекция, проводимая автором кода при завершении разработки каждого класса. Этот этап включал проверку именования переменных и методов, наличие комментариев к сложным участкам кода, а также общую логическую структуру классов. Также проводилась формальная инспекция отдельных модулей с привлечением руководителя практики, который проверял корректность реализации поставленных задач и соответствие кода принятым стандартам.

#### **5.1.2. Стандарты кодирования, применявшиеся в проекте.**

При разработке приложения использовались стандарты кодирования, принятые для языка C# и платформы .NET. Основное внимание уделялось следующим правилам. Имена классов, методов и свойств должны быть написаны в стиле PascalCase,

например, `TaskManager`, `GetAllTasks`, `IsImportant`. Имена закрытых полей класса оформляются в стиле `_camelCase` с подчёркиванием в начале, например, `_tasks` для списка задач. Имена локальных переменных оформляются в стиле `camelCase` без подчёркивания, например, `taskName`, `dueDate`. Асинхронные методы, если бы они использовались, должны иметь суффикс «`Async`». Все методы, которые могут выбрасывать исключения, имеют соответствующую документацию с указанием типа исключения. Все классы содержат краткое описание своего назначения в виде XML-комментариев. Кроме того, в ходе инспектирования проверялось, что все методы имеют минимально необходимую сложность, не содержат дублирующегося кода и используют встроенные средства языка (например, LINQ-запросы) вместо ручной обработки коллекций.

### 5.1.3. Программные средства для проведения инспекции.

Инспектирование кода выполнялось с использованием встроенных средств Visual Studio 2022. При написании кода анализаторы Roslyn автоматически проверяли файлы проекта и отображали предупреждения о нарушениях правил в окне «Список ошибок». Это позволяло сразу видеть, какие строки кода требуют доработки, и исправлять их непосредственно в процессе разработки. При завершении работы над каждым классом выполнялся дополнительный анализ через пункт меню «Анализ» и «Запустить анализ кода», который проверял код на соответствие правилам безопасности и надёжности. Все выявленные замечания исправлялись. Например, в процессе инспектирования было выявлено, что в некоторых методах используются имена переменных, не соответствующие правилам стиля, что было исправлено путём переименования. Также проверялись все классы на наличие обработки возможных исключений, корректность использования модификаторов доступа и наличие документации. По итогам инспектирования весь код библиотеки `TaskManager.Library` был приведён к единому стилю, что обеспечило его читаемость и упростило дальнейшую поддержку и развитие приложения.

## ЗАКЛЮЧЕНИЕ

В результате прохождения учебной практики с 8 по 28 июня 2024 г. были закреплены теоретические знания и расширены профессиональные умения.

По итогам практики получены следующие результаты:

1. Разработано приложение для управления задачами «Менеджер задач», реализующее полный набор функций для работы со списком задач: добавление, просмотр, редактирование, удаление, фильтрация по статусу, поиск по названию и описанию, сортировка по приоритету и сроку выполнения, пометка задач как «Важных», отображение статистики, а также сохранение и загрузка данных в форматах JSON и XML.

2. Разработана динамическая библиотека классов TaskManager.Library, содержащая всю бизнес-логику приложения, включая классы для управления задачами (Manager), фильтрации (Filter), сортировки (Sorter), поиска (Searcher), сбора статистики (Stats) и работы с файлами (Storage). Библиотека построена с использованием принципов объектно-ориентированного программирования и LINQ.

3. Создан пользовательский интерфейс в виде WPF-приложения TaskManager.WPF, включающий главное окно со списком задач, окна для добавления и редактирования задач, а также окно статистики. Реализована удобная навигация и визуальное выделение важных и просроченных задач.

4. Написаны модульные тесты для всех классов библиотеки с использованием MSTest. Общее количество тестов составило 57, все тесты успешно проходят, что подтверждает корректность работы основных функций приложения.

5. Произведена отладка программного модуля с использованием встроенных средств Visual Studio (пошаговое выполнение, точки останова, окна наблюдения). В процессе отладки выявлены и устранены следующие проблемы: отсутствие автосохранения при закрытии приложения (добавлено

сохранение в tasks.json), несоответствие названий статусов в фильтре и перечислении (приведены к единому формату), а также проблемы с временными файлами в тестах (добавлен метод [TestCleanup] для их удаления).

6. Проведено инспектирование кода на соответствие стандартам кодирования C#. Проверялись правила именования (PascalCase для классов и методов, \_camelCase для закрытых полей), использование модификаторов доступа, обработка исключений и наличие XML-комментариев. Все выявленные нарушения были исправлены, код приведён к единому стилю.

По окончании практики был получен практический опыт разработки требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

## СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Коршунова А. В. Разработка приложений на платформе .NET с использованием C# и WPF : учебное пособие. – Москва : МГТУ им. Н.Э. Баумана, 2024. – 180 с. – ISBN 978-5-7038-6123-5.
2. Кривоносова Н. В. Технология WPF. Разработка модулей программного обеспечения : практикум. – Санкт-Петербург : СПбГУТ им. М.А. Бонч-Бруевича, 2021. – 132 с.
3. Фримен А. Программирование на C# для профессионалов. – Москва : Эксмо, 2023. – 800 с. – ISBN 978-5-04-187342-4.
4. Шилдт Г. C# 10.0. Полное руководство. – Москва : Диалектика, 2023. – 1008 с. – ISBN 978-5-907515-07-5.
5. Савицкий Н. И. LINQ. Язык интегрированных запросов в C# : учебное пособие. – Москва : Инфра-М, 2024. – 180 с. – ISBN 978-5-16-019776-0.
6. Хорев П. Б. Объектно-ориентированное программирование с примерами на C# : учебное пособие. – Москва : ФОРУМ, 2023. – 200 с. – ISBN 978-5-00091-680-3.
7. Hawthorn A. Desktop Applications with C#: Building WPF and Windows Forms Solutions. – Independently Published, 2025. – 206 p. – ISBN 979-8288475344.
8. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. – Москва : Стандартинформ, 2022. – 12 с.
9. ГОСТ 7.0.100-2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – Москва : Стандартинформ, 2022. – 120 с.
10. ГОСТ Р 7.0.5-2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления. – Москва : Стандартинформ, 2022. – 36 с.



## Task.cs

```

using System;
namespace TaskManager
{
    public class Task
    {
        public Guid Id { get; set; }
        public string Name { get; set; }
        public string Description { get; set; }
        public Priority1 Priority { get; set; }
        public DateTime DueDate { get; set; }
        public Status1 Status { get; set; }
        public bool IsImportant { get; set; }
        public string ImportantMark => IsImportant ? "*" : "";
        public Task()
        {
            Id = Guid.NewGuid();
            Status = Status1.Новая;
            Priority = Priority1.Средний;
            DueDate = DateTime.Now.AddDays(7);
        }
        public Task(string name, string description, Priority1 priority,
            DateTime dueDate, Status1 status, bool isImportant = false)
        {
            Id = Guid.NewGuid();
            Name = name;
            Description = description;
            Priority = priority;
            DueDate = dueDate;
            Status = status;
            IsImportant = isImportant;
        }

        public enum Priority1
        {
            Низкий,
            Средний,
            Высокий
        }
        public enum Status1
        {
            Новая,
            ВПроцессе,
            Завершена
        }
    }
}

```

## Manager.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using TaskManager.Library;

namespace TaskManager.Library.Services
{
    public class Manager
    {
        private List<Task> _tasks = new List<Task>();

        public List<Task> GetAll()
        {
            return _tasks.ToList();
        }

        public Task GetById(Guid id)
        {
            return _tasks.FirstOrDefault(t => t.Id == id);
        }

        public void Add(Task task)
        {
            if (task == null)
                throw new ArgumentNullException(nameof(task));

            _tasks.Add(task);
        }

        public void AddRange(List<Task> tasks)
        {
            if (tasks == null)
                throw new ArgumentNullException(nameof(tasks));

            _tasks.AddRange(tasks);
        }

        public void Edit(Task updatedTask)
        {
            if (updatedTask == null)
                throw new ArgumentNullException(nameof(updatedTask));

            var existingTask = GetById(updatedTask.Id);
            if (existingTask == null)
                throw new KeyNotFoundException($"Задача с ID {updatedTask.Id} не найдена");

            existingTask.Name = updatedTask.Name;
            existingTask.Description = updatedTask.Description;
            existingTask.Priority = updatedTask.Priority;
            existingTask.DueDate = updatedTask.DueDate;
            existingTask.Status = updatedTask.Status;
        }
    }
}
```

```

        existingTask.IsImportant = updatedTask.IsImportant;
    }

    public void Delete(Guid id)
    {
        var task = GetById(id);
        if (task == null)
            throw new KeyNotFoundException($"Задача с ID {id} не найдена");

        _tasks.Remove(task);
    }

    public void Clear()
    {
        _tasks.Clear();
    }
}

```

### Filter.cs

```

using System.Collections.Generic;
using System.Linq;
using static TaskManager.Task;

namespace TaskManager.Library.Services
{
    public class Filter
    {
        public List<Task> FilterByStatus(List<Task> tasks, Status1 status)
        {
            return tasks.Where(t => t.Status == status).ToList();
        }

        public List<Task> FilterByStatuses(List<Task> tasks, List<Status1> statuses)
        {
            return tasks.Where(t => statuses.Contains(t.Status)).ToList();
        }

        public List<Task> FilterByImportant(List<Task> tasks)
        {
            return tasks.Where(t => t.IsImportant).ToList();
        }

        public List<Task> FilterByPriority(List<Task> tasks, Priority1 priority)
        {
            return tasks.Where(t => t.Priority == priority).ToList();
        }
    }
}

```

### Searcher.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using TaskManager.Library;

namespace TaskManager.Library.Services
{
    public class Searcher
    {
        public List<Task> Search(List<Task> tasks, string query)
        {
            if (string.IsNullOrEmpty(query))
                return tasks.ToList();

            query = query.ToLower().Trim();

            return tasks.Where(t =>
                t.Name.ToLower().Contains(query) ||
                t.Description.ToLower().Contains(query)
            ).ToList();
        }

        public List<Task> SearchByExactName(List<Task> tasks, string name)
        {
            if (string.IsNullOrEmpty(name))
                return tasks.ToList();

            return tasks.Where(t =>
                t.Name.Equals(name, StringComparison.OrdinalIgnoreCase)
            ).ToList();
        }
    }
}

```

## Sorter.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using TaskManager.Library;

namespace TaskManager.Library.Services
{
    public class Sorter
    {
        public List<Task> SortByPriority(List<Task> tasks)
        {
            return tasks.OrderByDescending(t => t.Priority).ToList();
        }

        public List<Task> SortByDueDate(List<Task> tasks)
        {

```

```

        return tasks.OrderBy(t => t.DueDate).ToList();
    }

    public List<Task> SortByPriorityAndDueDate(List<Task> tasks)
    {
        return tasks
            .OrderByDescending(t => t.Priority)
            .ThenBy(t => t.DueDate)
            .ToList();
    }
}

```

## Stats.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using TaskManager.Library;
using static TaskManager.Task;

namespace TaskManager.Library.Services
{
    public class Stats
    {
        public Dictionary<Status1, int> GetStatusCounts(List<Task> tasks)
        {
            var result = new Dictionary<Status1, int>();

            foreach (Status1 status in Enum.GetValues(typeof(Status1)))
            {
                result[status] = tasks.Count(t => t.Status == status);
            }

            return result;
        }

        public int GetOverdueCount(List<Task> tasks)
        {
            return tasks.Count(t => t.DueDate < DateTime.Now && t.Status !=
Status1.Завершена);
        }

        public int GetTotalCount(List<Task> tasks)
        {
            return tasks.Count;
        }

        public Dictionary<string, int> GetAllStats(List<Task> tasks)
        {
            var stats = new Dictionary<string, int>
            {
                ["Всего задач"] = GetTotalCount(tasks),
            }
        }
    }
}

```

```

        ["Просрочено"] = GetOverdueCount(tasks)
    };

    var statusCounts = GetStatusCounts(tasks);
    foreach (var kvp in statusCounts)
    {
        stats[kvp.Key.ToString()] = kvp.Value;
    }

    return stats;
}

public bool IsOverdue(Task task)
{
    if (task == null) return false;
    return task.DueDate < DateTime.Now && task.Status != Task.Status1.Завершена;
}
}
}

```

## Storage.cs

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Text.Json;
using System.Xml.Serialization;
using TaskManager.Library;

namespace TaskManager.Library.Services
{
    public class Storage
    {
        public void SaveToJson(List<Task> tasks, string filePath)
        {
            var options = new JsonSerializerOptions
            {
                WriteIndented = true
            };
            var json = JsonSerializer.Serialize(tasks, options);
            File.WriteAllText(filePath, json);
        }

        public List<Task> LoadFromJson(string filePath)
        {
            if (!File.Exists(filePath))
                throw new FileNotFoundException($"Файл {filePath} не найден");

            var json = File.ReadAllText(filePath);
            return JsonSerializer.Deserialize<List<Task>>(json) ?? new List<Task>();
        }

        public void SaveToXml(List<Task> tasks, string filePath)

```

```

    {
        var serializer = new XmlSerializer(typeof(List<Task>));
        using var writer = new StreamWriter(filePath);
        serializer.Serialize(writer, tasks);
    }

    public List<Task> LoadFromXml(string filePath)
    {
        if (!File.Exists(filePath))
            throw new FileNotFoundException($"Файл {filePath} не найден");

        var serializer = new XmlSerializer(typeof(List<Task>));
        using var reader = new StreamReader(filePath);
        return (List<Task>)serializer.Deserialize(reader) ?? new List<Task>();
    }

    public List<Task> LoadFromFile(string filePath)
    {
        var extension = Path.GetExtension(filePath).ToLower();
        return extension switch
        {
            ".json" => LoadFromJson(filePath),
            ".xml" => LoadFromXml(filePath),
            _ => throw new NotSupportedException($"Формат {extension} не
поддерживается")
        };
    }

    public void SaveToFile(List<Task> tasks, string filePath)
    {
        var extension = Path.GetExtension(filePath).ToLower();
        switch (extension)
        {
            case ".json":
                SaveToJson(tasks, filePath);
                break;
            case ".xml":
                SaveToXml(tasks, filePath);
                break;
            default:
                throw new NotSupportedException($"Формат {extension} не
поддерживается");
        }
    }
}

```